

NLP Deliverable 1

Quora Question Pairs — Duplicate Detection

Joel Sanfeliu

Carlos Raso

Pau Anguera

Universitat de Barcelona
Natural Language Processing
April 30, 2026

Submitted to: `daniel.ortiz@ub.edu`

Contents

1	Introduction	2
2	Dataset and Data Splits	2
3	Baseline Model	2
3.1	Feature Representation	2
3.2	Classifier	2
3.3	Limitations	2
4	Engineered Features	3
4.1	Token-Level Jaccard Similarity (Joel Sanfeliu)	3
4.1.1	Definition	3
4.1.2	Rationale	3
4.1.3	Empirical Validation	3
4.2	Length and Word-Overlap Features (Joel Sanfeliu)	3
4.3	TF-IDF Cosine Similarity (Carlos Raso)	4
4.3.1	TF-IDF Vectorization	4
4.3.2	Cosine Similarity (from scratch)	4
4.3.3	Advantage over Jaccard	4
4.3.4	Empirical Validation	4
4.4	Character 3-gram Similarity (Pau Anguera)	4
4.4.1	Definition	4
4.4.2	Motivation	5
4.4.3	Empirical Validation	5
5	Improved Model	5
5.1	Feature Combination	5
5.2	Classifier	5
6	Results	5
6.1	Evaluation Protocol	5
6.2	Summary Table	5
6.3	Analysis	6
7	Work Distribution	6

Introduction

The Quora Question Pairs challenge requires determining whether two questions posted on Quora are semantically equivalent (i.e. duplicates). This deliverable presents a complete machine-learning pipeline addressing that binary classification task. We implement a **baseline** model based on Bag-of-Words features and an **improved** model that enriches the feature set with multiple hand-crafted and distributional similarity measures, all combined into a single Logistic Regression classifier.

The report is organized as follows: Section 2 describes the dataset and splits; Section 3 details the baseline approach; Section 4 explains each engineered feature; Section 5 presents the improved model; Section 6 reports and analyses the evaluation metrics; and Section 7 summarises the work distribution.

Dataset and Data Splits

The dataset is the publicly available Quora Question Pairs corpus. Because it is a Kaggle challenge without labelled test data, we create our own fixed splits using the following procedure (random seed 123 throughout):

1. A 95 / 5 stratified split yields the *train+val* set and the **test set**.
2. The train+val portion is further split 95 / 5 to give the **training set** and the **validation set**.

The resulting split sizes are shown in Table 1.

Table 1: Dataset splits used throughout the project.

Split	Samples	Share
Training	291,897	$\approx 90.25\%$
Validation	15,363	$\approx 4.75\%$
Test	16,172	$\approx 5.00\%$
Total	323,432	100%

All vectorizers and models are fitted *exclusively* on the training split to prevent any form of data leakage.

Baseline Model

Feature Representation

The baseline uses a **Bag-of-Words** (BoW) representation. A `CountVectorizer` with unigrams is fitted on all training questions (both `question1` and `question2`), yielding a vocabulary of **74,825** tokens. Each question pair is represented by the horizontal concatenation of the two BoW vectors, producing a sparse feature matrix of shape $(n, 149,650)$.

Classifier

A Logistic Regression (LR) model with the `liblinear` solver and `random_state=123` is trained on these features. The decision threshold is kept at the default value of 0.5.

Limitations

The BoW baseline suffers from several known shortcomings:

- **No semantic distance:** the two question vectors are concatenated but never compared — the model learns association weights independently, not the similarity between the pair.
- **Vocabulary mismatch:** paraphrases that use different words (“automobile” vs. “car”) score zero overlap.
- **Ignores word importance:** common words like *what* or *how* receive the same weight as informative content words.
- **No character-level robustness:** typos or slight morphological variations break exact word matching.

Engineered Features

Three additional similarity features are implemented from scratch, each assigned to one group member.

Token-Level Jaccard Similarity (Joel Sanfeliu)

Definition

Given two strings, let A and B be the *sets of lower-case alphanumeric tokens* obtained from each string. The Jaccard similarity is:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

The function returns 0 when both sets are empty. It is implemented entirely from scratch — no `sklearn` or `scipy` helper functions are used.

Rationale

Jaccard is insensitive to document length (unlike raw word-overlap counts) and has a natural interpretation: $J = 0$ means no shared tokens; $J = 1$ means identical token sets. Duplicate questions tend to share most of their vocabulary, so high Jaccard scores are a strong signal.

Empirical Validation

On a random sample of 500 training pairs:

Table 2: Jaccard similarity statistics by class.

Subset	Mean	Std
All pairs	0.364	0.236
Duplicates	0.465	—
Non-duplicates	0.301	—

The clear separation between classes confirms the feature’s discriminative power.

Length and Word-Overlap Features (Joel Sanfeliu)

Four hand-crafted features capture structural similarity between question pairs:

The `common_word_ratio` shows a mean of 0.544 for duplicate pairs vs. 0.361 for non-duplicates on the 500-sample evaluation.

Table 3: Length and overlap features.

Feature	Description
<code>len_diff</code>	$ len(q_1) - len(q_2) $ at character level
<code>word_count_diff</code>	$ words(q_1) - words(q_2) $
<code>common_word_ratio</code>	$ \text{tokens}(q_1) \cap \text{tokens}(q_2) / \max(\text{words}(q_1), \text{words}(q_2))$
<code>len_ratio</code>	$\min(len(q_1), len(q_2)) / \max(len(q_1), len(q_2))$

TF-IDF Cosine Similarity (Carlos Raso)

TF-IDF Vectorization

A `TfidfVectorizer` with unigrams and bigrams (`ngram_range=(1,2)`), `sublinear_tf=True`, and a maximum vocabulary of **50,000** features is fitted on the training questions. The larger bigram vocabulary captures local phrase structure beyond single-word matching.

Cosine Similarity (from scratch)

Given two sparse row vectors $\mathbf{u}$ and $\mathbf{v}$:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$$

The implementation operates directly on `scipy.sparse` matrices (dot product and norms via `.dot()` and `.toarray()`), with a guard against zero-norm vectors. For the full dataset, vectorised numpy operations are used to compute all cosine scores in a single pass, avoiding a Python-level loop.

Advantage over Jaccard

TF-IDF down-weights high-frequency function words (*what, how, the*) and up-weights rare, informative content words. Two questions sharing the word *photosynthesis* receive a larger similarity boost than two questions both containing *what*, unlike Jaccard which treats all tokens equally.

Empirical Validation

Table 4: TF-IDF cosine similarity statistics by class (500-sample evaluation).

Subset	Mean cosine
Duplicates	0.477
Non-duplicates	0.328

Character 3-gram Similarity (Pau Anguera)

Definition

Let $G_n(s)$ be the set of unique character n -grams of string s (with $n = 3$ by default). The character-level Jaccard similarity is:

$$C(s_1, s_2) = \frac{|G_3(s_1) \cap G_3(s_2)|}{|G_3(s_1) \cup G_3(s_2)|}$$

Both strings are lower-cased before extraction. Empty strings return 0.

Motivation

Word-level methods (Jaccard, BoW, TF-IDF) fail when questions contain typos or morphological variants. Character n -grams decompose words into overlapping sub-strings, making the similarity measure robust to such noise. For example, *python* and *pyton* share zero words but share the 3-grams *pyt*, *yth*/*yto*, *tho*/*ton*.

Empirical Validation

Table 5: Character 3-gram similarity statistics by class (500-sample evaluation).

Subset	Mean similarity
Duplicates	0.464
Non-duplicates	0.297

The feature provides approximately 56% higher average similarity for duplicate pairs, and handles typos that break word-level matching (e.g. “How do I learn Python?” vs. “How do I lern Python?” yields a similarity of 0.58 despite having no words in common after the typos).

Improved Model

Feature Combination

The improved feature matrix horizontally concatenates all individual feature blocks:

$$\mathbf{X}_{\text{all}} = [\mathbf{X}_{\text{BoW}} \mid \mathbf{x}_{\text{Jaccard}} \mid \mathbf{x}_{\text{TF-IDF cos}} \mid \mathbf{X}_{\text{length}} \mid \mathbf{x}_{\text{char-3g}}]$$

The resulting matrix has shape $(n, 149,657)$: 149,650 BoW dimensions plus 7 engineered features (1 Jaccard, 1 TF-IDF cosine, 4 length, 1 character n -gram). All matrices are stored as `scipy.sparse.csr_matrix` for memory efficiency.

Classifier

The same Logistic Regression architecture is used (`liblinear`, `C=1.0`, `random_state=123`, `max_iter=200`), allowing a fair comparison with the baseline.

Results

Evaluation Protocol

Models are evaluated using three metrics from `sklearn.metrics`:

- **ROC-AUC** (`roc_auc_score`): threshold-independent measure of ranking quality.
- **Precision** (`precision_score`): fraction of predicted duplicates that are true duplicates.
- **Recall** (`recall_score`): fraction of true duplicates that the model retrieves.

Probabilities are obtained via `predict_proba`, and a decision threshold of 0.5 is used for precision/recall.

Summary Table

Table 6: Evaluation results for both models across all splits.

Model	Split	ROC-AUC	Precision	Recall
Baseline (BoW + LR)	Train	0.8899	0.7820	0.6867
	Val	0.8046	0.6774	0.6106
	Test	0.8137	0.6952	0.6186
Improved (All features + LR)	Train	0.9211	0.7979	0.7701
	Val	0.8670	0.7287	0.6949
	Test	0.8712	0.7319	0.6975

Analysis

Baseline. The BoW + LR model achieves a test ROC-AUC of 0.8137. The gap between training (0.8899) and validation (0.8046) AUC indicates moderate overfitting, expected given the high-dimensional sparse input with no explicit regularisation tuning.

Improved model. Adding the seven engineered features increases every metric substantially:

- **ROC-AUC:** +0.0575 on the test set (+7.1% relative), reaching 0.8712.
- **Recall:** +0.0789 on the test set (+12.8% relative) — the model now captures a much larger fraction of actual duplicate pairs.
- **Precision:** +0.0367 on the test set (+5.3% relative).

The train–test gap for the improved model (AUC: 0.9211 → 0.8712) is comparable to the baseline, suggesting the engineered features do not introduce additional overfitting. The key improvement is that the similarity-based features give the model a direct signal about *how similar* the two questions are, rather than forcing it to learn this relationship implicitly from paired word occurrences.

Work Distribution

Table 7: Contributions by group member.

Member	Contributions
Joel Sanfeliu	Implemented <code>jaccard_similarity</code> (from scratch), <code>get_jaccard_features</code> , and <code>get_length_features</code> . Documented usage and class-wise analysis in <code>utils_StudentA.ipynb</code> .
Carlos Raso	Implemented <code>fit_tfidf_vectorizer</code> , <code>cosine_similarity_sparse</code> (from scratch), and <code>get_tfidf_cosine_features</code> . Documented usage and comparative analysis in <code>utils_StudentB.ipynb</code> .
Pau Anguera	Implemented <code>char_ngram_similarity</code> (from scratch) and <code>get_char_ngram_features</code> . Documented the character-level approach and typo-robustness analysis in <code>utils_StudentC.ipynb</code> .
All members	Jointly responsible for <code>train_models.ipynb</code> , <code>reproduce_results.ipynb</code> , <code>utils.py</code> (shared utilities: data splits, BoW vectorizer, feature assembly, evaluation, model I/O), and the <code>environment.yml</code> file.